

AI OPs Assignment 1 Question 2

Madhav Arun Krishna - DA24B012

August 2026

1. Screenshot attached.

Experiments > iris-classifier >

Runs

Views

Group by

metrics.rmse < 1 and params.model = "tree"

All time

State: Active

Datasets

Sort: Run Name

Columns

New run

		Metrics								Parameters								
	Run Name	Created	Dataset	Dr	Src	Model	accuracy	f1_macro	train_loss	val_accuracy	Use	Batch Size	Epochs	Hidden Layer Sizes	Learning Rate	lr	Ve	De
	mlp-1		-				0.75942...	0.7541...	0.653064...	0.75542857142...	m	128	25	(128.)	0.1	-	-	-
	mlp-2		-				0.97314...	0.9729...	0.042741...	0.971	m	128	25	(128.)	0.01	-	-	-
	mlp-3		-				0.97685...	0.9767...	0.008338...	0.97742857142...	m	128	25	(128.)	0.001	-	-	-
	mlp-4		-				0.95885...	0.9584...	0.049791...	0.95942857142...	m	128	25	(32.)	0.01	-	-	-
	mlp-5		-				0.964	0.9638...	0.052186...	0.96642857142...	m	128	25	(64.)	0.01	-	-	-
	mlp-6		-				0.97257...	0.9723...	0.059467...	0.97142857142...	m	128	25	(256.)	0.01	-	-	-

2. Analysis:

I tried varying two hyperparameters, learning rate and MLP width. The first three runs hold the width constant and vary the learning rate (0.1, 0.01 and 0.001). I observed that the models at 0.01 and 0.001 performed nearly the same, and much better than the one at 0.1. This suggests that a learning rate of 0.1 is simply too great for the model to settle and leads to very unstable learning.

As for the model width (I ran it for 32, 64 and 256), we can see that increasing the width from 32 to 64 provided negligible benefits whereas increasing it further to 256 seemed to give more substantial benefits. However, this was still worse than the best run from the learning rate experiments where we had kept our width fixed to 128. This leads us to believe that for this dataset, beyond a certain width, there is not much benefit.

Additionally, on analysing the train loss and val acc curves, we saw no clear evidence of overfitting. Apart from the learning rate = 0.1 experiment, which had a val acc curve that kept decreasing and a train loss curve that was fluctuating without decreasing (indicating that the learning rate was too high), all the other experiments had normal train loss and val acc curves.

The overall best run was the one with width 128 and learning rate 0.001, since it achieved the highest validation accuracy. Overall, learning rate had a larger effect on performance than width (beyond a threshold).

3. I added the following logging lines:

```
mlflow.log_param("Hidden Layer Sizes", hidden_layer_sizes)
mlflow.log_param("Learning Rate Init", learning_rate_init)
mlflow.log_param("Batch Size", batch_size)
mlflow.log_param("Epochs", epochs)
for epoch in range(epochs):
    mlflow.log_metric("train_loss", model.loss_, step=epoch)
    mlflow.log_metric("val_accuracy", accuracy_score(y_val, val_preds), step=epoch)
```

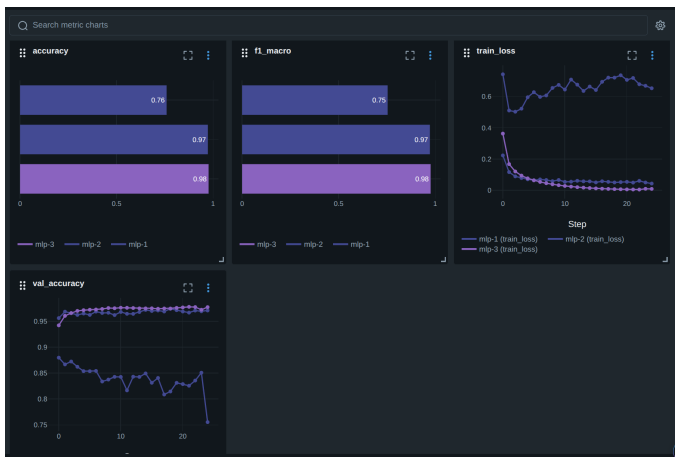


Figure 1: Learning rate changes

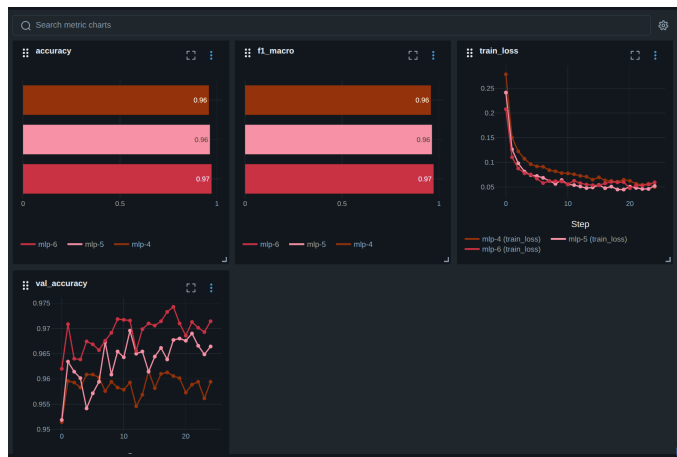


Figure 2: Width changes